

M.Sc. DATA SCIENCE & BIG DATA ANALYTICS
SEMESTER III — PRACTICAL / ASSIGNMENT

TASK 3

**Dataset Preparation and Preprocessing for
Deep Learning**

Submitted By

Name: Satyam Krishnakumar Yadav

Roll No.: 45

Student ID: 5662854

Submitted To

Faculty Name: Usha Nandhini

Department: Computer Science

Academic Year: 2026–2027

1. Introduction

This report documents my implementation of Task 3: Dataset Preparation and Preprocessing for Deep Learning. The objective of the task was to prepare an image dataset for deep learning applications through exploratory data analysis, data-quality checking, preprocessing, augmentation, feature transformation, and train-validation-test splitting.

I used the CIFAR-10 dataset because it is a standard image dataset containing 10 object categories. The completed notebook loads the dataset, examines its structure and class distribution, checks data quality, performs preprocessing and label transformation, creates training/validation/test partitions, and demonstrates image augmentation.

The submitted Colab output confirms that the implementation completed the preprocessing workflow successfully. The notebook does not train a CNN or other predictive model; model training is outside the stated deliverables of this dataset-preparation task.

2. Objective

To prepare an image dataset for deep learning applications through exploratory data analysis, data-quality checking, preprocessing, augmentation, feature transformation, and train-validation-test splitting.

3. Dataset Description

The CIFAR-10 dataset contains 60,000 colour images of size 32×32 pixels belonging to 10 classes. The TensorFlow/Keras loading output in the notebook shows 50,000 training images and 10,000 test images, with three RGB channels.

Class ID	Class Name
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Deer
5	Dog
6	Frog
7	Horse
8	Ship
9	Truck

The original training set contains 5,000 images per class and the original test set contains 1,000 images per class. This indicates an approximately balanced dataset.

```
print("Number of Classes:", len(class_names))

print("\nImage Data Type:", X_train_full.dtype)
print("Minimum Pixel Value:", X_train_full.min())
print("Maximum Pixel Value:", X_train_full.max())
```

```
===== DATASET INFORMATION =====
Total Training Samples: 50000
Total Testing Samples: 10000
Image Height: 32
Image Width: 32
Number of Channels: 3
Number of Classes: 10

Image Data Type: uint8
Minimum Pixel Value: 0
Maximum Pixel Value: 255
```

Exploratory Data Analysis

EDA is performed to understand the structure, distribution, quality, and characteristics of the dataset before preprocessing.

```
# Convert labels to 1D array

y_train_flat = y_train_full.flatten()
y_test_flat = y_test.flatten()

# Count samples in each class
train_class_counts = np.bincount(y_train_flat)
test_class_counts = np.bincount(y_test_flat)

# Create DataFrame
class_distribution = pd.DataFrame({
    "Class": class_names,
    "Training Samples": train_class_counts,
    "Testing Samples": test_class_counts
})

class_distribution
```

	Class	Training Samples	Testing Samples	
0	Airplane	5000	1000	
1	Automobile	5000	1000	
2	Bird	5000	1000	
3	Cat	5000	1000	
4	Deer	5000	1000	
5	Dog	5000	1000	
6	Frog	5000	1000	
7	Horse	5000	1000	
8	Ship	5000	1000	
9	Truck	5000	1000	

Next steps: [Generate code with class_distribution](#) [New interactive sheet](#)

```
plt.figure(figsize=(12, 6))

plt.bar(class_names, train_class_counts)

plt.title("CIFAR-10 Training Dataset Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of Images")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Figure: Class distribution and EDA output

4. Tools and Technologies Used

Tool / Library	Purpose
Google Colab	Notebook execution and experiment environment
Python	Implementation language
NumPy	Array operations and data-quality checks
Pandas	Tabular summaries and dataset analysis
Matplotlib	EDA and visualization
Seaborn	Visualization library imported for analysis
TensorFlow / Keras	CIFAR-10 dataset access and image augmentation
Scikit-learn	Stratified train-validation split

5. Preprocessing Workflow

The implemented workflow can be summarized as:

1. Load CIFAR-10 dataset
2. Inspect dimensions, data type, pixel range, and class labels
3. Perform exploratory data analysis and class-distribution analysis
4. Check missing, infinite, duplicate, and inconsistent-dimension records
5. Analyze pixel and RGB-channel distributions
6. Create stratified training and validation sets while keeping the test set separate
7. Convert image arrays to float32 and normalize pixels from 0–255 to 0–1
8. One-hot encode the class labels
9. Create an image augmentation pipeline
10. Visualize original, augmented, and normalized images
11. Summarize the final prepared dataset and data quality

6. Exploratory Data Analysis

The EDA stage was used to understand the structure and distribution of the dataset before applying transformations.

6.1 Dataset Structure

Property	Observed Value
Original training images	50,000
Original test images	10,000
Image dimensions	$32 \times 32 \times 3$
Image data type	uint8
Pixel range before preprocessing	0–255
Number of classes	10

6.2 Class Distribution

The class-distribution analysis shows 5,000 training images and 1,000 test images for every class. The corresponding bar chart in the notebook shows approximately equal class representation.

```
print("Number of Classes:", len(class_names))

print("\nImage Data Type:", X_train_full.dtype)
print("Minimum Pixel Value:", X_train_full.min())
print("Maximum Pixel Value:", X_train_full.max())
```

```
===== DATASET INFORMATION =====
Total Training Samples: 50000
Total Testing Samples: 10000
Image Height: 32
Image Width: 32
Number of Channels: 3
Number of Classes: 10

Image Data Type: uint8
Minimum Pixel Value: 0
Maximum Pixel Value: 255
```

Exploratory Data Analysis

EDA is performed to understand the structure, distribution, quality, and characteristics of the dataset before preprocessing.

```
# Convert labels to 1D array

y_train_flat = y_train_full.flatten()
y_test_flat = y_test.flatten()

# Count samples in each class
train_class_counts = np.bincount(y_train_flat)
test_class_counts = np.bincount(y_test_flat)

# Create DataFrame
class_distribution = pd.DataFrame({
    "Class": class_names,
    "Training Samples": train_class_counts,
    "Testing Samples": test_class_counts
})

class_distribution
```

	Class	Training Samples	Testing Samples	
0	Airplane	5000	1000	
1	Automobile	5000	1000	
2	Bird	5000	1000	
3	Cat	5000	1000	
4	Deer	5000	1000	
5	Dog	5000	1000	
6	Frog	5000	1000	
7	Horse	5000	1000	
8	Ship	5000	1000	
9	Truck	5000	1000	

Next steps: [Generate code with class_distribution](#) [New interactive sheet](#)

```
plt.figure(figsize=(12, 6))

plt.bar(class_names, train_class_counts)

plt.title("CIFAR-10 Training Dataset Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of Images")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

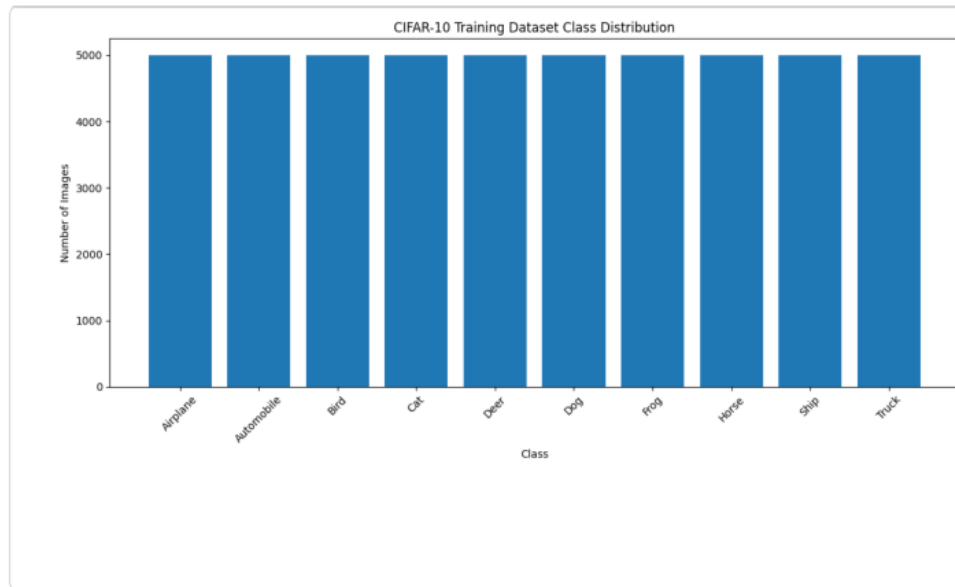
Figure: Class distribution and EDA output

6.3 Image Visualization

Sample images from all ten classes and a set of randomly selected images were visualized. This provides a direct visual check of the image content and labels.

9/25/26, 4:18 PM

Task 3: Dataset Preparation and Preprocessing for Deep Learning - Colab



Observation

The class distribution is approximately balanced. Each class contains nearly the same number of images, which reduces the risk of severe class imbalance during deep learning model training.

```
# Display sample images from each class
plt.figure(figsize=(15, 8))

for class_id in range(10):
    # Find first image belonging to the class
    index = np.where(y_train_flat == class_id)[0][0]

    plt.subplot(2, 5, class_id + 1)
    plt.imshow(X_train_full[index])
    plt.title(class_names[class_id])
    plt.axis("off")

plt.suptitle("One Sample Image from Each CIFAR-10 Class", fontsize=16)
plt.tight_layout()
plt.show()
```

https://colab.research.google.com/drive/1cbax49MN5YjQTOXSpuYrdzfvw_ZZloK#scrollTo=N1nr-A2yLiCO&printMode=true

4/16

Figure: Sample images from each CIFAR-10 class

One Sample Image from Each CIFAR-10 Class



```
# Display random images

plt.figure(figsize=(15, 10))

random_indices = np.random.choice(len(X_train_full), 20, replace=False)

for i, index in enumerate(random_indices):
    plt.subplot(4, 5, i + 1)

    plt.imshow(X_train_full[index])
    plt.title(class_names[y_train_flat[index]])
    plt.axis("off")

plt.suptitle("Random CIFAR-10 Images", fontsize=16)

plt.tight_layout()
plt.show()
```

Figure: Random CIFAR-10 images

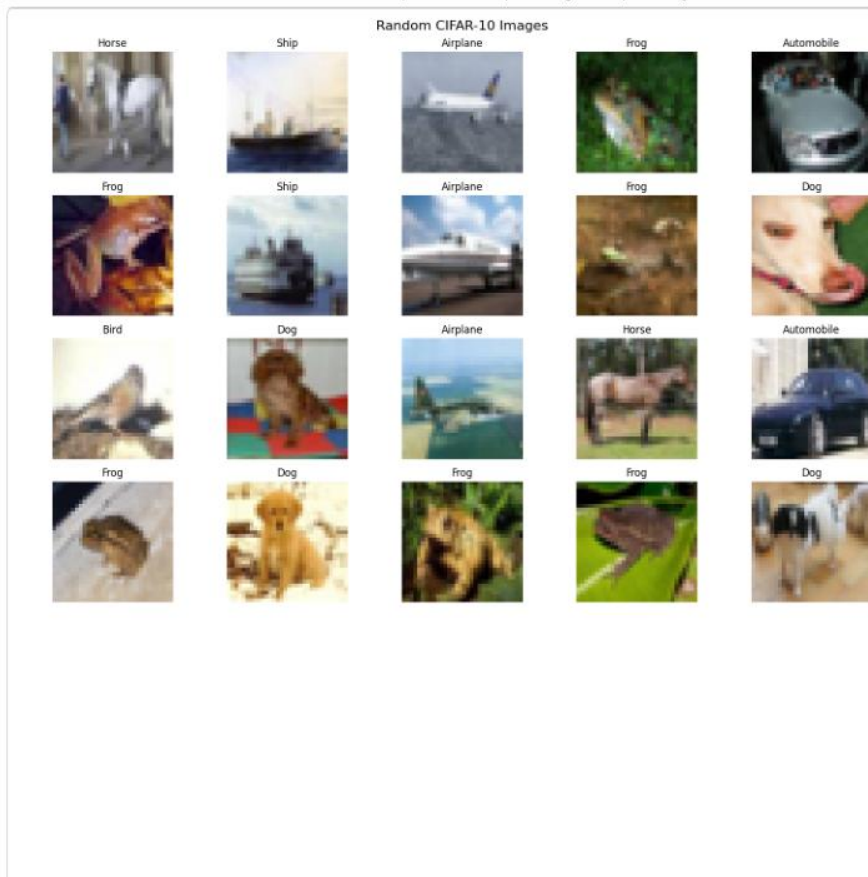
7. Data Quality Checking

7.1 Missing Values

The notebook checks NaN values in both images and labels. The result was zero missing values in training images, testing images, training labels, and testing labels. Therefore, no missing-value imputation was necessary.

9/25/26, 4:18 PM

Task 3: Dataset Preparation and Preprocessing for Deep Learning - Colab



10. Missing Data Analysis

Since CIFAR-10 is stored as numerical image arrays, missing values can be checked using NumPy.

A missing pixel value would normally appear as NaN.

```
print("Missing values in training images:", np.isnan(X_train_full).sum())
print("Missing values in testing images:", np.isnan(X_test).sum())

print("Missing values in training labels:", np.isnan(y_train_full).sum())
print("Missing values in testing labels:", np.isnan(y_test).sum())
```

```
Missing values in training images: 0
Missing values in testing images: 0
Missing values in training labels: 0
Missing values in testing labels: 0
```

Observation

No missing values were detected in the CIFAR-10 image arrays or labels. Therefore, no missing-value imputation was required.

```
print("Infinite values in training images:",
      np.isinf(X_train_full).sum())
```

https://colab.research.google.com/drive/1cbax49MIN5YjQTOXSpUyrdzFVw_ZZIoK#scrollTo=N1nr-A2yLiCO&printMode=true

6/16

Figure: Missing-data and data-quality checks

7.2 Infinite Values

Infinite-value checks were performed on training and testing image arrays. Both returned zero infinite values.

7.3 Duplicate Image Check

An exact duplicate check was performed on a sample of 10,000 training images. All 10,000 were unique in the checked sample, resulting in zero potential exact duplicates.

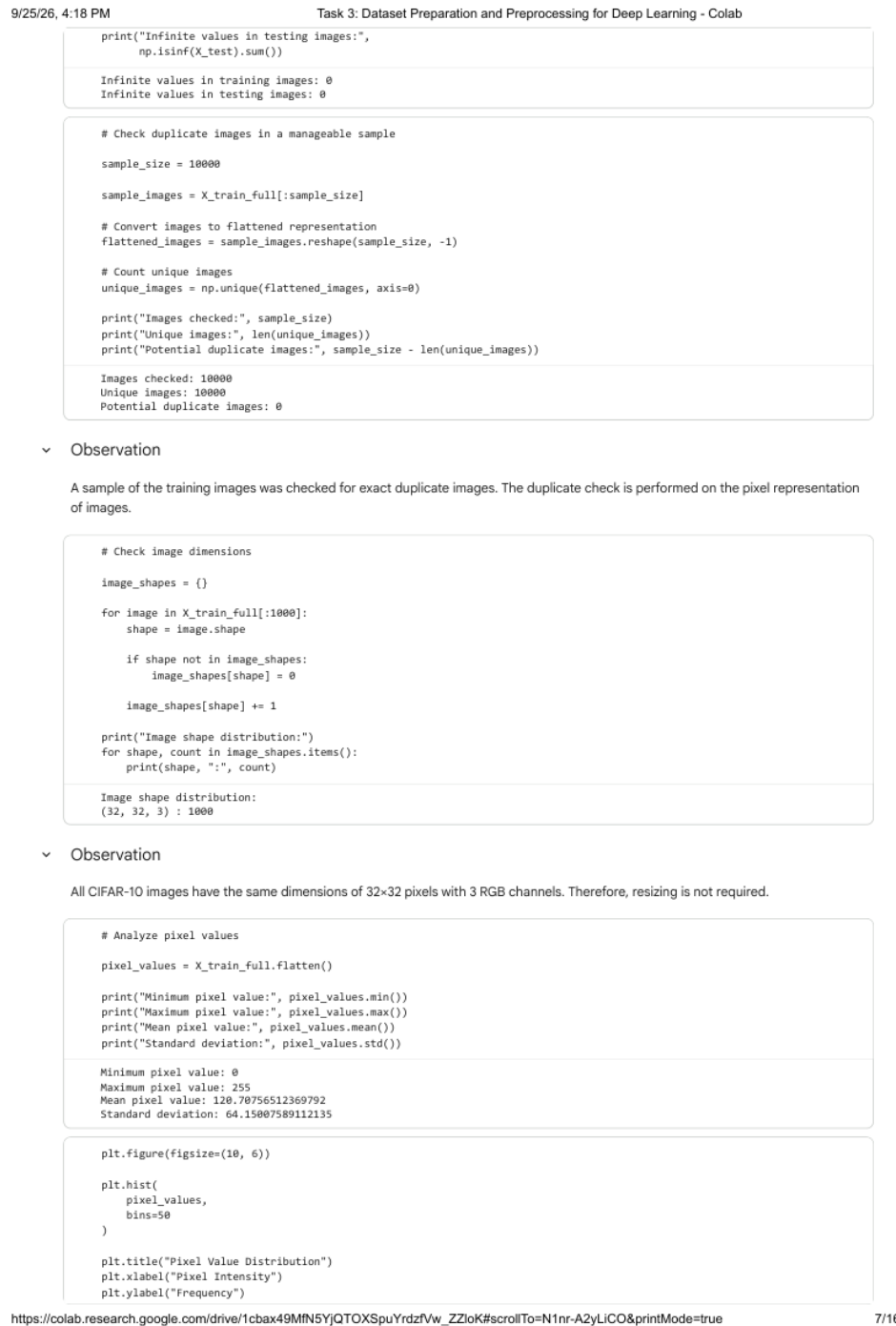


Figure: Duplicate and image-dimension checks

7.4 Image Dimension Consistency

A sample of 1,000 images was checked for shape consistency. All 1,000 had the shape (32, 32, 3). Because the dimensions are consistent, resizing was not required.

8. Pixel and Feature Analysis

Pixel values were analyzed before normalization. The minimum value was 0, the maximum was 255, the mean was approximately 120.71, and the standard deviation was approximately 64.15.

The RGB channel means were approximately 125.31 for Red, 122.95 for Green, and 113.87 for Blue. The notebook also visualizes the overall pixel-value distribution and average RGB-channel intensities.

```
# Analyze pixel values

pixel_values = X_train_full.flatten()

print("Minimum pixel value:", pixel_values.min())
print("Maximum pixel value:", pixel_values.max())
print("Mean pixel value:", pixel_values.mean())
print("Standard deviation:", pixel_values.std())

Minimum pixel value: 0
Maximum pixel value: 255
Mean pixel value: 120.70756512369792
Standard deviation: 64.15007589112135

plt.figure(figsize=(10, 6))

plt.hist(
    pixel_values,
    bins=50
)

plt.title("Pixel Value Distribution")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

olab.research.google.com/drive/1cbax49MFN5YjQTOXSpuYrdzFVw_ZZloK#scrollTo=N1nr-A2yLiCO&printMode=true 7/16

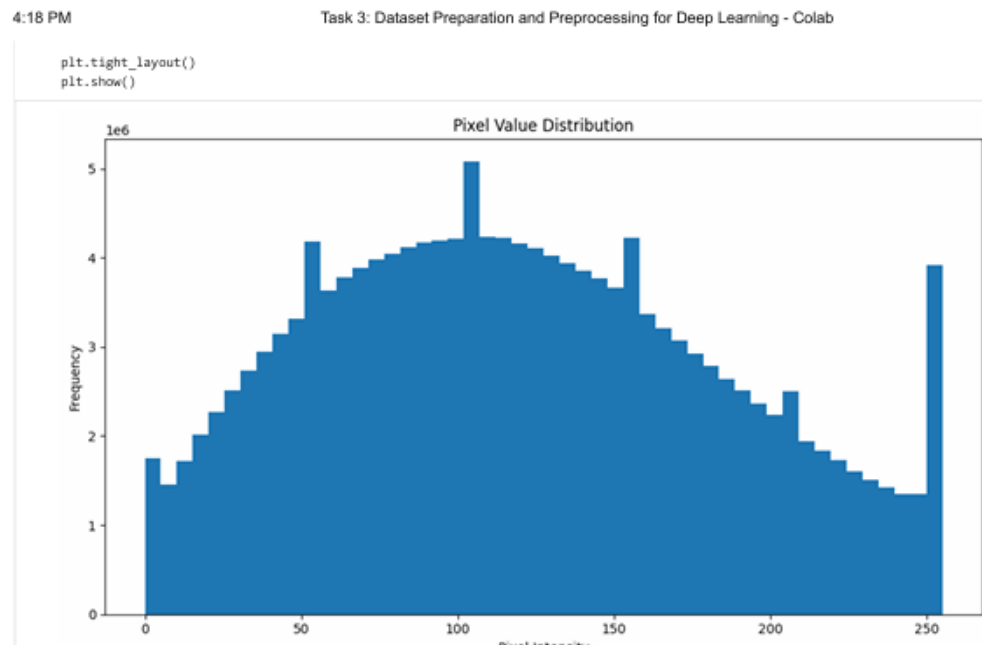


Figure: Pixel distribution and RGB-channel analysis

9. Train-Validation-Test Split

The original 50,000-image training set was divided using a stratified 90:10 split. The official CIFAR-10 test set was kept separate and was not included in the training/validation split.

Dataset	Number of Images	Label Shape
Training	45,000	(45,000, 1) before encoding
Validation	5,000	(5,000, 1) before encoding
Testing	10,000	(10,000, 1) before encoding

After the split, each class contains 4,500 training images, 500 validation images, and 1,000 test images. The use of stratification preserved the balanced class distribution across the partitions.

1, 9, 10 FIVE

100% 0. Validation

	Class	Train	Validation	Test	
0	Airplane	4500	500	1000	
1	Automobile	4500	500	1000	
2	Bird	4500	500	1000	
3	Cat	4500	500	1000	
4	Deer	4500	500	1000	
5	Dog	4500	500	1000	
6	Frog	4500	500	1000	
7	Horse	4500	500	1000	
8	Ship	4500	500	1000	
9	Truck	4500	500	1000	

Figure: Train-validation-test split

10. Image Normalization

The image arrays were converted to float32 and divided by 255.0. This transformed the pixel range from 0–255 to 0–1. The final training mean was approximately 0.4732 and the standard deviation was approximately 0.2515.

Stage	Pixel Range
Before normalization	0–255
After normalization	0–1

We will normalize them to:

0 to 1

```
# Convert image arrays to float32
X_train = X_train.astype("float32")
X_val = X_val.astype("float32")
X_test = X_test.astype("float32")

# Normalize pixel values to 0-1
X_train_normalized = X_train / 255.0
X_val_normalized = X_val / 255.0
X_test_normalized = X_test / 255.0

print("Original pixel range:")
print(X_train.min(), "to", X_train.max())

print("\nNormalized pixel range:")
print(X_train_normalized.min(), "to", X_train_normalized.max())
```

Original pixel range:
0.0 to 255.0

Normalized pixel range:
0.0 to 1.0

```
#Verify Normalization
print("Training data:")
print("Minimum:", X_train_normalized.min())
print("Maximum:", X_train_normalized.max())
print("Mean:", X_train_normalized.mean())
print("Standard Deviation:", X_train_normalized.std())

print("\nValidation data:")
print("Minimum:", X_val_normalized.min())
print("Maximum:", X_val_normalized.max())

print("\nTesting data:")
print("Minimum:", X_test_normalized.min())
print("Maximum:", X_test_normalized.max())
```

Training data:
Minimum: 0.0
Maximum: 1.0
Mean: 0.4732333
Standard Deviation: 0.2514992

Validation data:
Minimum: 0.0
Maximum: 1.0

Testing data:
Minimum: 0.0
Maximum: 1.0

✓ Label Encoding

For deep learning classification, labels can be represented using one-hot encoding.

```
# One-hot encode labels

y_train_encoded = to_categorical(y_train, num_classes=10)
y_val_encoded = to_categorical(y_val, num_classes=10)
y_test_encoded = to_categorical(y_test, num_classes=10)

print("Original label shape:", y_train.shape)
print("Encoded label shape:", y_train_encoded.shape)

print("\nExample:")
print("Original:", y_train[0])
print("Encoded:", y_train_encoded[0])
```

Original label shape: (45000, 1)
Encoded label shape: (45000, 10)

Example:
Original: [1]
Encoded: [0. 1. 0. 0. 0. 0. 0. 0. 0.]

Figure: Normalization and label encoding



Before vs After Preprocessing

```
plt.figure(figsize=(10, 5))

# Original
plt.subplot(1, 2, 1)

plt.imshow(X_train[0].astype("uint8"))

plt.title(
    "Before Normalization\nPixel Range: 0-255"
)

plt.axis("off")

# Normalized
plt.subplot(1, 2, 2)

plt.imshow(X_train_normalized[0])

plt.title(
    "After Normalization\nPixel Range: 0-1"
)

plt.axis("off")

plt.tight_layout()
plt.show()
```

Figure: Original versus augmented image

11. Label Encoding

The class labels were converted from integer labels to one-hot encoded vectors using Keras `to_categorical`. The training labels changed from shape (45,000, 1) to (45,000, 10), representing the ten CIFAR-10 classes.

For example, the automobile class (label 1) is represented as [0, 1, 0, 0, 0, 0, 0, 0, 0, 0].

Label Encoding

For deep learning classification, labels can be represented using one-hot encoding.

```
1 # One-hot encode labels
2
3 y_train_encoded = to_categorical(y_train, num_classes=10)
4 y_val_encoded = to_categorical(y_val, num_classes=10)
5 y_test_encoded = to_categorical(y_test, num_classes=10)
6
7 print("Original label shape:", y_train.shape)
8 print("Encoded label shape:", y_train_encoded.shape)
9
10 print("\nExample:")
11 print("Original:", y_train[0])
12 print("Encoded:", y_train_encoded[0])
13
14 Original label shape: (45000, 1)
15 Encoded label shape: (45000, 10)
16
17 Example:
18 Original: [1]
19 Encoded: [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Figure: Normalization and label encoding

12. Image Augmentation

An `ImageDataGenerator` was created to produce variations of training images. The augmentation configuration used in the submitted notebook is:

Augmentation	Setting
Rotation	15 degrees
Width shift	10%
Height shift	10%
Horizontal flip	Enabled
Zoom	10%

The notebook generated and displayed multiple augmented versions of a sample image. The outputs show visually modified versions while retaining the original class content.

9/25/26, 4:18 PM

Task 3: Dataset Preparation and Preprocessing for Deep Learning - Colab

23. Image Augmentation

Image augmentation artificially creates variations of existing training images. This can improve model generalization and reduce overfitting.

The following transformations are applied:

- Random rotation
- Horizontal translation
- Vertical translation
- Horizontal flipping
- Zoom

```
# Create ImageDataGenerator for augmentation

augmentation_generator = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.10,
    height_shift_range=0.10,
    horizontal_flip=True,
    zoom_range=0.10
)

print("Image augmentation generator created successfully!")

Image augmentation generator created successfully!
```

Visualize Augmented Images

```
# Select one image

sample_image = X_train_normalized[0]
sample_label = y_train.flatten()[0]

# Reshape image to create a batch
sample_image_batch = np.expand_dims(sample_image, axis=0)

# Generate augmented images
augmented_images = augmentation_generator.flow(
    sample_image_batch,
    batch_size=1,
    shuffle=False
)

plt.figure(figsize=(15, 8))

for i in range(10):

    augmented_image = next(augmented_images)[0]

    plt.subplot(2, 5, i + 1)
    plt.imshow(augmented_image)
    plt.title("Augmented")
    plt.axis("off")

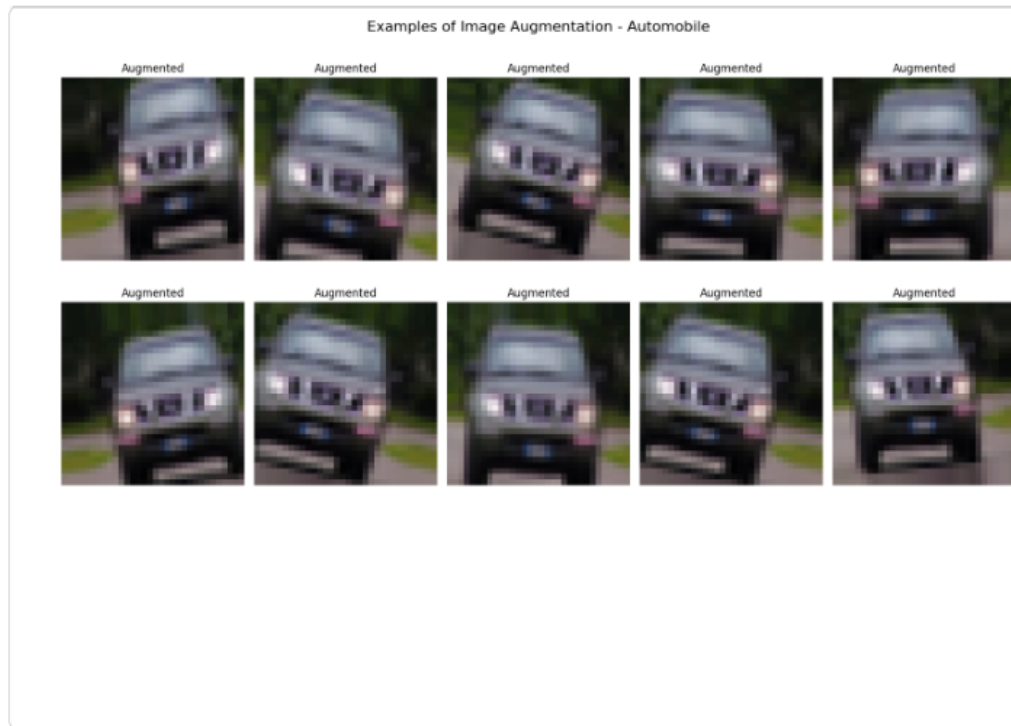
plt.suptitle(
    "Examples of Image Augmentation - " + class_names[sample_label],
    fontsize=16
)

plt.tight_layout()
plt.show()
```

https://colab.research.google.com/drive/1cbax49MfN5YjQTOXSpuYrdzfVw_ZZIoK#scrollTo=N1nr-A2yLiCO&printMode=true

12/16

Figure: Image augmentation implementation



▼ Original vs Augmented Image

```
plt.figure(figsize=(12, 5))

# Original image
plt.subplot(1, 2, 1)

plt.imshow(X_train_normalized[0])

plt.title(
    "Original Image\n" + class_names[y_train.flatten()][0]
)

plt.axis("off")

# Augmented image
augmented_image = next(
    augmentation_generator.flow(
        np.expand_dims(X_train_normalized[0], axis=0),
        batch_size=1,
        shuffle=False
    )
)[0]

plt.subplot(1, 2, 2)

plt.imshow(augmented_image)

plt.title("Augmented Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Figure: Augmented image visualization

13. Final Prepared Dataset

Dataset	Images	Image Shape	Encoded Label Shape
Training	45,000	(45,000, 32, 32, 3)	(45,000, 10)
Validation	5,000	(5,000, 32, 32, 3)	(5,000, 10)
Testing	10,000	(10,000, 32, 32, 3)	(10,000, 10)

The final data-quality summary reported zero missing values and zero infinite values, image dimensions of 32×32 with three channels, ten classes, and a normalized pixel range of 0–1.

```
=====
FINAL EDA AND PREPROCESSING REPORT
=====
...

1. DATASET
Dataset: CIFAR-10

2. ORIGINAL DATASET
Training samples: 50000
Testing samples: 10000

3. FINAL SPLIT
Training: 45000
Validation: 5000
Testing: 10000

4. IMAGE INFORMATION
Image size: 32 x 32
Channels: RGB
Classes: 10

5. DATA QUALITY
Missing values: 0
Infinite values: 0

6. PREPROCESSING
Pixel normalization: 0-255 → 0-1
Label encoding: One-hot encoding

7. AUGMENTATION
Rotation: 15 degrees
Width shift: 10%
Height shift: 10%
Horizontal flip: Enabled
Zoom: 10%

8. FINAL PIXEL RANGE
Minimum: 0.0
Maximum: 1.0

=====
PREPROCESSING COMPLETED SUCCESSFULLY
=====
```



Figure: Final dataset shapes and data-quality summary

14. Observations and Findings

- The CIFAR-10 dataset was successfully loaded with 50,000 training images and 10,000 test images.
- Each image has a consistent shape of $32 \times 32 \times 3$ and uses RGB channels.
- All ten classes have equal representation in the original dataset.
- No missing values were found in the image arrays or labels.
- No infinite pixel values were detected.
- The checked sample of 10,000 training images contained no exact duplicate images.
- Pixel values originally ranged from 0 to 255 and were normalized to 0–1.
- Labels were transformed into one-hot encoded vectors for ten classes.
- A stratified split created 45,000 training images, 5,000 validation images, and 10,000 test images.
- Image augmentation was implemented using rotation, shifting, horizontal flipping, and zoom.
- Visualization confirmed the effect of augmentation and normalization on the images.
- The resulting dataset is structured and preprocessed for subsequent deep learning model development.

15. Limitations

- The duplicate check was performed on a sample of 10,000 training images rather than all 50,000 training images.
- The notebook checks numerical data quality and image-shape consistency; it does not perform file-level image corruption recovery because CIFAR-10 is loaded as validated numerical arrays through Keras.
- No deep learning model was trained in this Task 3 implementation because the submitted task focuses on dataset preparation and preprocessing.
- The preprocessed arrays are demonstrated in memory in the notebook. A separate exported compressed dataset file is not evidenced in the submitted Colab PDF.

16. Conclusion

I successfully completed the dataset preparation and preprocessing workflow for the CIFAR-10 dataset. The work covered exploratory data analysis, class distribution analysis, missing and numerical quality checks, duplicate and dimension checks, pixel analysis, train-validation-test splitting, normalization, label encoding, image augmentation, and visualization.

The final dataset consists of 45,000 training images, 5,000 validation images, and 10,000 test images. The images are represented as 32×32 RGB arrays, the pixel values are normalized to the 0–1 range, and labels are one-hot encoded into ten classes. The augmentation pipeline provides additional image variations through rotation, shifts, horizontal flipping, and zoom.

Based on the completed implementation and outputs, the prepared dataset is ready to be used as input for a CNN or transfer-learning based deep learning model in a subsequent task.

17. References

- CIFAR-10 dataset accessed through TensorFlow/Keras (`cifar10.load_data()`).
- TensorFlow / Keras documentation for `ImageDataGenerator` and `to_categorical`.
- Scikit-learn documentation for `train_test_split` and stratified splitting.
- The submitted Google Colab notebook and its generated outputs for Task 3.

18.Git Hub Link:-

<https://github.com/228263satyam/TASK-3-Dataset-Preparation-and-Preprocessing-for-Deep-Learning.git>